RetailOps forecaster

A study guide for the team: how our ML model works, what rules it follows, and why every choice was made

CS2105 · IIT Patna · Semester III · Lump 1 (Data & Forecast)

State as of 31 Oct 2026: Phases A–E built and gated, 163 tests passing on the laptop, frozen model + forecasts.parquet handed to the order model.

How to read this. The guide goes from simple to deep, in seven levels. Stop at any level and you will still have a correct picture, just a less detailed one.

Level 1 — the 2-minute version · Level 2 — key ideas in plain words · Level 3 — the workflow, step by step · Level 4 — the rules it never breaks · Level 5 — every decision and why · Level 6 — deep dive into the code · Level 7 — results, known limits and viva answers.

If you only have 10 minutes: read Levels 1–3 and the limits in Level 7.

Sources: docs/decisions.md, docs/design-principles.md, plan/forecaster-build-plan.md, plan/forecast-training-plan.md, plan/build-roadmap.md, config.yaml, forecast/*.py, outputs/forecast/ (scores, report, model manifest).

Contents

What the forecaster is 3

The one big idea . . . 3

In five sentences . . . 3

The ideas you need first 4

1. What is a quantile (a "line")? . . . 4

2. How does a model learn a quantile? Pinball loss . . . 4

3. Plan 7 days, act on 1 . . . 4

4. No peeking at the future . . . 4

5. One global model . . . 4

6. What is "good"? Beat an honest bar . . . 4

How a forecast is made, start to finish 6

The time windows . . . 7

Where the forecast goes: the handoff . . . 7

The rules the forecaster never breaks 8

The most important rule: R2, the 7-day gap . . . 8

All twelve rules . . . 8

Other house rules . . . 9

Every decision, with its reason 10

A. What the model is . . . 10

B. Fairness . . . 10

C. Data cleaning . . . 10

D. Inputs . . . 11

E. Training and checking . . . 11

F. Engineering and scope . . . 12

Inside the model 13

The inputs (24 features) . . . 13

The four models and their settings . . . 13

Post-processing, in this exact order . . . 14

Freezing and guards (freeze.py, predict.py) . . . 14

Code map . . . 14

How well it works, and where it is weak 16

Validation scores (all four lines) . . . 16

Known limits (say these openly in the viva) . . . 17

Likely viva questions, short answers . . . 17

Glossary . . . 18

LEVEL 1 · THE 2-MINUTE VERSION

What the forecaster is

RetailOps decides how much stock to order for a chain of **10 stores and 2 warehouses**, using real Walmart food sales (the M5 dataset, top 120 food items). It has two models working together:

- **The ML forecaster (this guide, Lump 1)** predicts how many units of each item each store will sell on each of the next 7 days.
- **The order model (Lump 2, our teammate's ILP)** takes those predictions and decides what to order, respecting cases, trucks, shelf space, budget and shelf life.

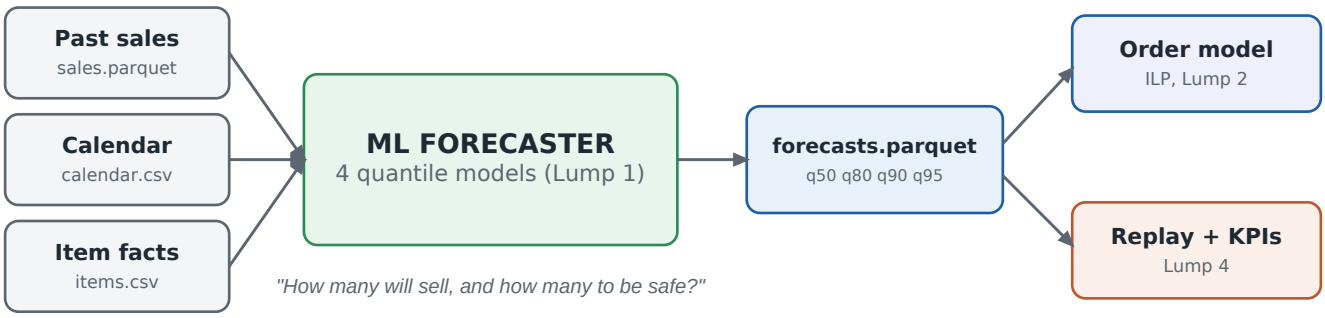


Figure 1. The forecaster sits between the raw data and the order model.

The one big idea

The forecaster does not give one number. For every store, item and day it gives **four lines**:

Line	Plain meaning	Who uses it
q50	The most likely amount (half the days sell less, half sell more)	Waste estimate, reports
q80	Enough for about 80 days out of 100	Slow sellers
q90	Enough for about 90 days out of 100	Most items
q95	Enough for about 95 days out of 100	Perishables and top 30 staples

Why? Because the order model needs to know *how much stock covers most days*, not just the average. An average forecast would leave it guessing how much safety stock to add.

In five sentences

- It is **four gradient-boosted tree models** (scikit-learn `HistGradientBoostingRegressor`), one per line, trained on all stores and items together.
- It learns from **recent sales, prices, the calendar and which item/store it is**.
- It obeys a strict **no-peeking rule**: every store-data input is at least 7 days old, so it could really have been known on the ordering morning.
- It **beats the teammate's simple rule** (4-week average) by about 10–12% in error on both validation periods, and every upper line beats it too.
- It was trained once, **frozen** with a manifest, and used to write `forecasts.parquet` (470,400 rows) for the 8-week test replay.

LEVEL 2 · KEY IDEAS IN PLAIN WORDS

The ideas you need first

1. What is a quantile (a "line")?

Imagine you looked at an item's sales on 100 similar days. Sort them. The 95th value is the **95% line (q95)**: only about 5 days in 100 sell more than that. If you stock to q95, you run out about 1 day in 20.

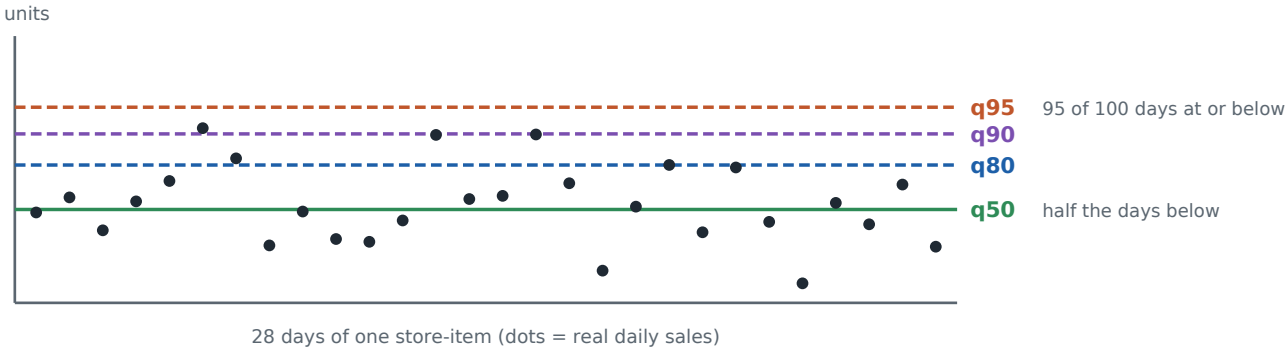


Figure 2. Each line is a level that real sales stay under a given share of the time.

2. How does a model learn a quantile? Pinball loss

Normal models punish being too high and too low equally. A quantile model punishes them **unequally**. For the 95% line:

- Line too **low** (we ran out): costs **0.95** per unit missed. Line 23, sales 25 → 0.95 × 2 = **1.9**.
- Line too **high** (spare stock): costs only **0.05** per unit over. Line 23, sales 15 → 0.05 × 8 = **0.4**.

So the model keeps pushing the line up until only about 5% of days go above it. Same idea for q80 (0.80 / 0.20) and q50 (0.5 / 0.5).

3. Plan 7 days, act on 1

Each morning (the **origin**) we forecast the next 7 days. The order model plans the whole week but only carries out day 1, then plans again tomorrow. Why forecast 7 days if we act on 1?

- **Supplier lead time is 5 days**: a warehouse order placed today arrives on day 5, so we must know day 5's demand today.
- **Batching**: cases, trucks and minimum orders make it cheaper to order for several days at once.
- **Perishables** (shelf life 2–7 days) must sell before they spoil, so we need the next few days.
- **Festivals**: the day before Thanksgiving is the biggest day of the year (~12,400 units vs ~9,000 nearby); stock must be ordered days before.

4. No peeking at the future

If the model ever uses a number it could not have known on the ordering morning, every result is fake. So the guiding question is: *what did the store manager know that morning?* This is turned into 12 rules (Level 4), each checked by an automatic test.

5. One global model

One item alone has only about 800 days of history, too little to learn festival spikes. So each model is trained on **all 10 stores × 120 items together**, with store, item, department and perishable flag as inputs. A quiet item borrows patterns (weekday shape, festival lift) from similar items.

6. What is "good"? Beat an honest bar

A score means nothing alone. We compare against two **baselines**:

- **Naive**: same day last week.
- **Stub**: the teammate's rule, average of the same weekday over the last 4 weeks. This is what the order model used before our model existed, so it is the honest bar.

The model must beat **both**, on **both** validation periods (one festive, one ordinary). It does:

MAE (units/day, lower is better)	Fold 1 (Nov–Dec 2014, festive)	Fold 2 (Jul–Oct 2015)
Naive baseline	4.88	5.21
Stub (teammate's 4-week rule)	4.39	4.72
Our model (q50)	**3.93** (10.4% lower than stub)	**4.15** (12.2% lower than stub)

MAE = mean absolute error: on average, how many units the q50 forecast is off by per store-item-day.

LEVEL 3 · THE WORKFLOW

How a forecast is made, start to finish

The forecaster is a **pipeline of small modules**, each with one job, joined by plain pandas tables. Every number lives in `config.yaml`.

green = builds the model · blue = choose settings and lock the model · orange = produce and judge output

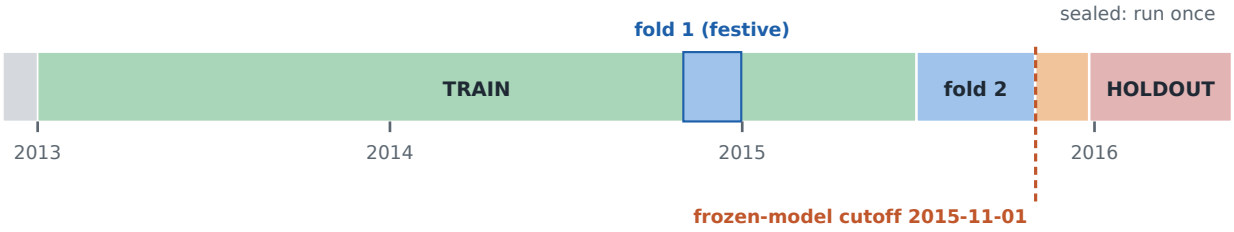


Figure 3. The ten steps and the file that does each one.

Step	What happens, in plain words
1. Load safely	Read config, sales, items and calendar. **Refuse** any sales file that reaches into the sealed holdout, and refuse any data on or after a model's cutoff.
2. Stats from the past	Anything computed over many rows (item codes, each item's normal price, which zero runs count as long) is computed on training rows only, with a required cutoff.
3. Pick training rows	Drop 25 December (stores closed). Long zero-sales runs are now kept (decision 26).
4. Build inputs	Make a dense grid (one row per store-item, one column per day). Read lags, averages and prices ending 7 days back; calendar for the target day; store/item codes. Stored as float32.
5. Train 4 models	One HistGradientBoostingRegressor with quantile loss for each of q50, q80, q90, q95, each with its own settings.
6. Post-process	Dead-item rule, calibration, sort the four lines so they never cross, clip at 0, set 25 Dec to 0.
7. Tune + calibrate	Try at most 6 settings on the two validation folds; check each line's coverage and stretch it if off by more than 3 points (no stretch was needed: all k = 1).
8. Freeze	Retrain once on everything before 2015-11-01 and save the model with a manifest.json (cutoff, features, settings, seed, versions, config hash). Laptop only.
9. Write forecasts	For each of the 56 test mornings (2015-11-01 → 12-26) forecast 7 days using only sales before that morning → forecasts.parquet.
10. Score + report	Compare with both baselines; test-window scores only after the replay.

The time windows

Data is split **by date**, never at random. A random split would put Tuesday in test and Wednesday in training, so the model would "know" the future.



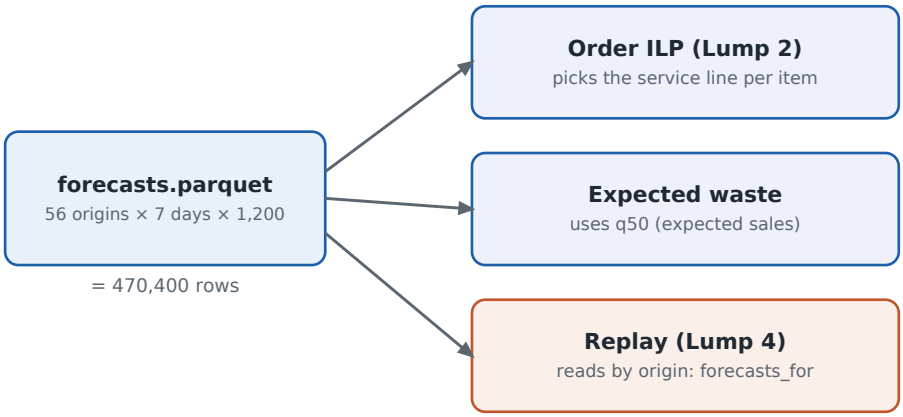
fold 1 trains on 2013-01 → 2014-10 only; fold 2 trains on 2013-01 → 2015-06. Final model trains on everything before 2015-11-01.

Figure 4. Every window is fixed in config.yaml (data.windows) before any training.

Window	Dates	Used for
Warm-up	2012-11-26 → 2012-12-31	History for the first lags only; never a target
Train	2013-01-01 → 2015-06-30	Fitting the models
Validation fold 1 (festive)	2014-11-01 → 2014-12-31	Choosing settings, calibration; has Thanksgiving + Christmas
Validation fold 2 (main)	2015-07-01 → 2015-10-31	Choosing settings, calibration
Test (development replay)	2015-11-01 → 2015-12-26	8-week replay; compare methods and layers
Holdout (sealed)	2015-12-27 → 2016-05-22	One final clean check, run exactly once after code freeze

Why both a test window and a holdout? The test replay is re-run for every layer and bug fix, so it slowly stops being "unseen". The holdout stays sealed through all of that. If its result is close to the test result, the gain is real.

Where the forecast goes: the handoff



Line each product is stocked to

perishable (20 items)	**q95**
top 30 staples	**q95**
slow (<5/day, 24 items)	**q80**
everything else	**q90**

Columns (fixed contract):

`origin, date, store_id, item_id, q50, q80, q90, q95`

Same as the stub file, so the teammate swaps it in with no code change.

Figure 5. forecasts.parquet is the only link between the forecaster and the order model.

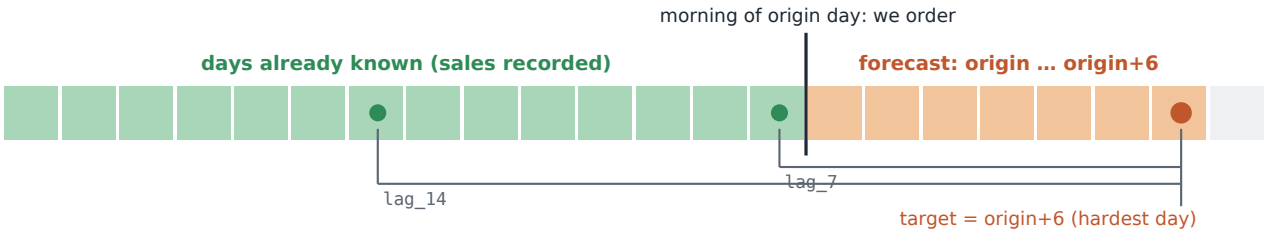
The order model reads the line each product is stocked to (the service line), orders only the gap between that line and the stock on hand, and uses q50 for the expected-waste cost of perishables. The replay reads the file one morning at a time with `forecast.forecasts_for(origin, cfg)`. The interface `forecast.predict(sales, origin)` computes the same rows live.

LEVEL 4 · THE RULES

The rules the forecaster never breaks

The rules come from `plan/forecast-training-plan.md` (R1–R12). Each important rule has a test in `tests/` that fails the build if it is broken. Nothing was trained until the fairness tests passed.

The most important rule: R2, the 7-day gap



Even for the farthest day (origin+6), lag_7 lands on origin−1 = yesterday, which is known. So the SAME inputs work for all 7 days, and the forecast for a date is identical no matter which morning asked for it. That is rule R2.

Figure 6. Why every store-data input is at least 7 days older than the day it predicts.

On the morning of day *o*, the hardest day to forecast is *o*+6. The newest data we have is from *o*−1, which is exactly 7 days before *o*+6. If every input is at least 7 days old, **one model serves the whole week**, and the forecast for a date is the same whichever morning asked. That property is also what makes the "future-poison" test possible.

The price of R2: a model allowed yesterday's sales would be about 12% better for tomorrow and about 5% better over the week (fold 2 MAE by days ahead: 3.62, 3.80, 3.97 ... 4.12 at day 7). But it needs ~7× the training rows and a rewrite of four tasks, and it breaks the one-model property the fairness tests rely on. **Team decision: keep R2.**

All twelve rules

Rule	In plain words	How it is enforced
R1 Locked windows	No training, tuning, product choice or cleaning step reads the test window or holdout.	load_sales refuses holdout rows; check_cutoff raises on rows after a cutoff. test_cutoff_guard_raises, test_dev_loader_never_returns_holdout_rows
R2 7-day rule	Every store-data input is at least 7 days older than the target day.	FEATURE_SOURCES registry; the grid never holds rows newer than target−7. test_future_poison_features, test_feature_age_by_perturbation, test_cross_item_perturbation
R3 Calendar is same-day	Weekday, month, events, SNAP, days to/since events, next event name, closed days may describe the target day (calendars are published in advance). Nothing else gets this exception.	test_calendar_features_ignore_sales (poisons all sales, calendar inputs must not move)
R4 Train-only statistics	Codes, normal prices, zero-run cut-offs, calibration factors come from training (or validation) rows only.	fit_encoders and zero_run_stats need a cutoff keyword. test_encoders_ignore_post_train_rows, test_long_zero_rule_uses_train_only
R5 Product choice from the past	The 120 items and 20 perishables are chosen on sales before the test window (a leak here was found and fixed).	test_selection_ignores_test_and_holdout_sales
R6 Tune on validation only	Try a few settings, pick by both validation folds, stop. Never look at test numbers.	test_tune_reads_no_test_rows

Rule	In plain words	How it is enforced
R7 Freeze before replay	Retrain once on train + validation, save with a manifest; the replay refuses a model whose cutoff is after the replay start.	load_frozen guards. test_frozen_guard_refuses_late_cutoff
R8 Day t sees only days before t	In the replay, each morning's forecast uses only sales before that morning; real sales are used only afterwards to score.	predict refuses an origin inside training data; write_forecasts reads sales before each origin
R9a Log every test run	Every test-window run is logged; reruns are allowed for layers and bug fixes, never to choose settings.	outputs/test_runs.log (planned, kept by the replay)
R9b Holdout once	After code freeze, run the holdout exactly once and report it as it comes out.	Process rule (planned: runs after code freeze)
R10 Report everything	Publish error and coverage for all items, perishables and slow sellers, even where the model loses. Never drop products after seeing results.	score always reports all / perishable / slow; error review lists the 20 worst store-items
R11 Same rules for every method	Baselines get the same cutoffs, starting stock and windows as our model.	Shared scorer and loader
R12 Reproducible	Seed fixed in config; same config → identical forecasts.	test_reproducible_same_seed

Other house rules

- **Every number in config.yaml**, no constants in code. The frozen model stores a hash of the config, so a changed config is detected.
- **Layers are switches, default off.** The core must pass all tests with every switch off.
- **Build in gated phases (A–E)**, stopping for review at 5 gates. Tests are written first for each task.
- **Don't break the teammate's inputs:** sales.parquet, items.csv, the stub files and the shared config sections are read, never written.
- **Missing means missing:** a day with no row, or a closed day, is NaN, never a fake 0.
- **Fail loudly:** bad input raises an error with a plain message instead of being trimmed or guessed.

Every decision, with its reason

Each row: what we chose, why, and what else we considered. Numbers in brackets are the decision numbers in `docs/decisions.md`, where the full evidence is.

A. What the model is

Decision	Chosen	Why	Alternatives considered
[1] Output	Four lines q50/q80/q90/q95 per store-item-day	Different items are stocked to different safety levels; the teammate's normal-curve 95% line covered only 81.7% of days before Thanksgiving 2014	One point forecast + safety-stock rule; mean + z × std
[2] Model type	HistGradientBoostingRegressor, quantile loss (scikit-learn 1.9.1)	Handles missing values natively, mixes numbers and categories, little tuning, trains in minutes, easy to explain	Small neural net (reserved); hand-written regression; linear quantile regression
[3] One or many models	One global model per line, all items and stores	One item has only ~800 days; pooling lets quiet items borrow patterns	1,200 per-item models; one per store
[4] Horizon	7 days, re-planned daily	5-day supplier lead time, batching, perishables, festival build-up	1 day (can't cover lead time); 14 days (too big for the solver)

B. Fairness

Decision	Chosen	Why	Alternatives considered
[5] R2 7-day gap	Kept	One model for the whole week; makes poison tests possible	Horizon-aware model (+5% over the week, ~7× rows, rewrite)
[6] Calendar	Same-day calendar allowed (R3)	Calendars are public in advance; hiding Thanksgiving would only hurt	7-day rule for calendar too
[7] Windows	Split by date; sealed holdout	Random splits leak the future	Random split
[8] Validation	Two folds: Nov–Dec 2014 + Jul–Oct 2015	Test window has festivals but fold 2 has none; fold 1 checks festive days without touching test	One fold; moving windows (would change shared files)
[9] Bar to beat	Naive + teammate's stub; must beat both on both folds	Beating only the weak bar proves little; the stub is called from her own code	Naive only

C. Data cleaning

Decision	Chosen	Why	Alternatives considered
[10] 25 December	Dropped from training, forecast 0, left out of scores, NaN in later inputs	Stores are closed; avoid learning a fake slump, stocking a shut store, or flattering scores	Leave 25 Dec as a zero day (tested slightly worse: Jan MAE 4.159 vs 4.141)
[11]→[26] Long zero runs	First dropped from training; now KEPT + dead-item rule	Error review showed items that stopped for good were the worst errors; the model must see 'a month of zeros, still zero'	Drop those items (R10 forbids); keep the original drop

Decision	Chosen	Why	Alternatives considered
[19] Slow sellers	Under 5 units/day (order model's rule), for reporting	Same groups the order model uses	Under 1/day (only 16 store-items)
[21] Calendar source	calendar.csv	Sales stop on 2015-12-26 but the last forecasts reach 2016-01-01	Calendar from sales file

D. Inputs

Decision	Chosen	Why	Alternatives considered
[13] Inputs used	Lags 7/14/21/28, 7- and 28-day means, 28-day spread, price inputs, momentum, store rank, calendar, codes	Each has a one-line reason (Level 6)	–
[13] next_event_name	Added	The biggest day (day before Thanksgiving) has no event of its own; knowing the next event separates run-ups from dips	Event flag only
[14] Same week last year	Left out	~40% of training rows have no year-ago value; getting older data would rebuild the shared sales file	–
[14] Price vs department; promo inputs	Dropped	No consistent gain on either fold (price vs dept is 99.5% just 'which item')	Keep them
[15] Price inflation	No fix needed	Prices rise ~1%/year; ratios and recent lags already carry the level; a 52-week normal price tested worse	52-week rolling normal price
[16] Horizon-aware model	Dropped	Too costly before the deadline; breaks one-model property	Optional layer
[17] Target-day price	Dropped	Prices change weekly on Saturdays; ~1% of days differ; worse on fold 1	Add it

E. Training and checking

Decision	Chosen	Why	Alternatives considered
[24] Keep rule	A change stays only if it helps BOTH folds; every try logged in experiments.csv	Many tries make lucky wins likely; two different periods make luck unlikely	One combined average score
[24] Every line vs baselines	q80/q90/q95 compared with both baselines by pinball	Orders use the upper lines, not q50	Check q50 only
[24] Error review + leak check	20 worst store-items; shuffle test; q50 MAE below 3.6 on fold 2 = suspected leak	Group averages hide bad items; input tests don't test the trained model	Input tests only
[25] Overfitting	Built-in early stopping off; tree count read from validation curve every 50 trees; 3-seed check	Built-in early stopping never fired; seed alone moves the score ~0.7%	Fixed 300 trees; one seed

Decision	Chosen	Why	Alternatives considered
[27] Recency weighting	Dropped	Recent level already in lags; would halve weight of the only earlier Christmas	Half-life 365/730 days
[28] Per-line settings	Each line has its own settings; upper lines lr 0.05	q50 best near 300 trees, q95 keeps improving to 450	One setting for all
[12]/[28] Calibration	Stretch the gap above q50 by k if coverage off > 3 pts; result k = 1 for all	A fixed unit shift over-stocks small sellers and under-stocks big ones	Unit shift; per-group factors
[20] Score the used line	cover_used_line	The line a product is stocked to is what decides empty shelves	Overall coverage only
[29] Freeze	Train once through 2015-10-31, manifest, laptop only	Laptop and cloud differ in the 3rd decimal; the handed-over file must be reproducible on the laptop	Freeze in cloud; retrain every origin

F. Engineering and scope

Decision	Chosen	Why	Alternatives considered
[18] Feature cache	Dropped; inputs rebuilt each run as float32	Building takes ~3 s; a cache bloats OneDrive and risks stale bugs	Cache keyed by cutoff + config + hash
[22] Layers	B after core; C and M only if asked; P7 hook off	Core must work alone	Build all
[23] Scope	Stock ordering only; use libraries	Small enough to finish and test	Staff shifts, routing, own algorithms (reserved)

LEVEL 6 · DEEP DIVE

Inside the model

The inputs (24 features)

All store-data inputs end at least 7 days before the target day. Feature names below are exactly the columns in the frozen manifest.

Group	Inputs	Why it helps
Recent sales	lag_7, lag_14, lag_21, lag_28	Sales on the same weekday in each of the last 4 weeks
Averages and spread	roll_mean_7, roll_mean_28, roll_std_28	The item's current level and how jumpy it is
Price	price_7, price_chg_4w, price_rel_item	Price a week back; change vs 4 weeks before; vs the item's normal price (catches discounts)
Trend	momentum = roll_mean_7 ÷ roll_mean_28	Is the item speeding up or slowing down?
Popularity	store_rank_28 (0–1)	Top sellers behave differently
Calendar (target day)	wday, month, is_event, event_name, snap, days_to_event, days_since_event, next_event_name	Weekday shape, festivals, food-stamp days, run-ups (capped at 30 days)
Who	store_id, item_id, dept_id, is_perishable	Lets one global model tell items apart

How inputs are built safely (features.py)

- **Dense grid:** one row per store-item, one column per calendar day. "7 days back" is a column offset, so a missing day can't shift the window.
- The grid's newest column is `newest target − 7`: newer sales rows are **never placed in the grid**, so the code physically cannot read them.
- **Missing stays missing:** the grid starts as NaN; late-launch items keep NaN early lags; 25 Dec units are NaN. Trees handle NaN natively.
- **Half-window rule:** averages use the days present, and are NaN only if fewer than half are present (4 of 7, 14 of 28).
- **Guarded divisions:** price_chg_4w = 0 and price_rel_item = 1 when a price is missing or 0; momentum = 1 when the 28-day mean is 0.
- Prices are summed in **whole cents** so averages are exact; a day's input never depends on which batch it was built in.

The four models and their settings

Each line has its own settings, chosen on both folds and confirmed on 3 seeds (seed fixed at 1 for the final model, min_samples_leaf 50 for all):

Line	Learning rate	Trees (max_iter)	Leaf nodes	Note
q50	0.1	300	31	The tuned version lost on fold 1, so q50 kept the starting settings
q80	0.05	150	63	
q90	0.05	600	31	
q95	0.05	450	63	Keeps improving with more trees

A gradient-boosted model is a sum of many small decision trees. Each new tree is fitted to fix what the trees before it got wrong, measured by the pinball loss of its line. The learning rate says how big each correction is; leaf nodes say how detailed each tree can be.

Pinball loss, written out

For quantile *q*, actual sales *y* and forecast *f*:

```
loss = q × (y − f) if y ≥ f (we ran short)
loss = (1 − q) × (f − y) if y < f (spare stock)
```

For q50 this is half the absolute error, which is why MAE = 2 × q50 pinball. Code: `forecast/metrics.py::pinball`.

Post-processing, in this exact order

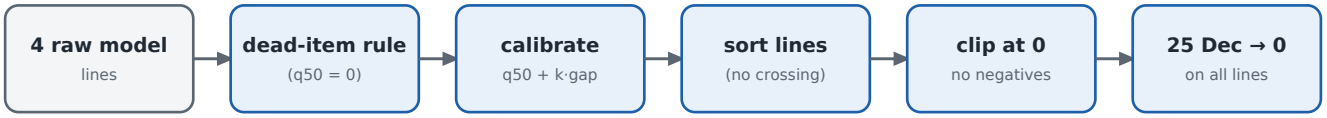


Figure 7. What happens to the raw model output before it is written.

- **Dead-item rule** (decision 26): if the 28-day average ending 7 days back is 0, set **q50 only** to 0. Upper lines keep the model's value, so an item that comes back still gets some stock.
- **Calibration:** new line = q50 + k × (line − q50), one k per line, fitted on validation only if coverage is off by more than 3 points (found by bisection between 0.5 and 4). Today every k = 1.
- **Sort** each row's four lines after calibration so q50 ≤ q80 ≤ q90 ≤ q95 always. **Clip** at 0. **25 Dec** → 0 on all four lines.

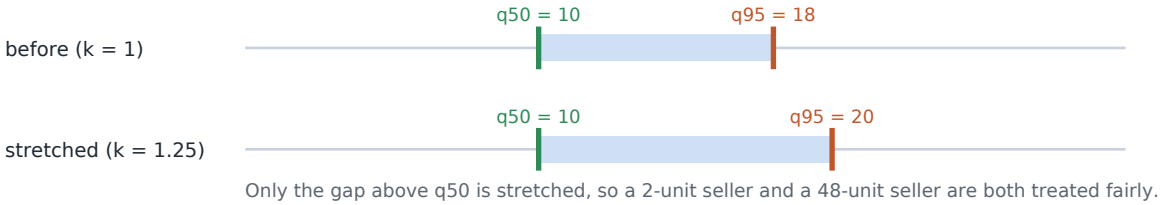


Figure 8. Calibration stretches the spread, it does not shift it.

Freezing and guards (freeze.py, predict.py)

`python -m forecast.freeze` retrains all four lines on 2013-01-01 → 2015-10-31 (1,226,019 rows, 126 s on the laptop) and saves `outputs/forecast/model/bundle.joblib` plus `manifest.json`:

```
{ "cutoff": "2015-11-01", "windows": {...}, "features": [24 names],
"quantiles": [0.5, 0.8, 0.9, 0.95], "params": {per line}, "shifts": {q50: 0, q80..q95: 1},
"seed": 1, "sklearn_version": "1.9.1", "config_sha256": "4910ad67…",
"platform": "Windows 11 / Python 3.14.7", "train_rows": 1226019 }
```

`load_frozen` refuses to load the model if: its cutoff is after the replay start, the config hash changed (forecast section + data windows, as sorted JSON, so comments and line endings don't matter), or the scikit-learn version differs. Freeze and write both refuse to run unless the computer is Windows (`freeze_platform`). A store-item with no sales before an origin would get the baseline (0 on every line); in the real run there were none.

Code map

File	Job
forecast/data.py	load_config, load_sales (refuses holdout), check_cutoff, val_folds, calendar
forecast/baseline.py	Naive and stub baselines (calls the teammate's own stubs.make_stubs.forecast_targets)
forecast/metrics.py	pinball, coverage, MAE, closed_mask, service_line, segments, score
forecast/evaluate.py	Score a method on both folds; log_try for the experiment log

File	Job
forecast/features.py	FEATURE_SOURCES, fit_encoders, zero_run_stats, training_rows, build_features, P7 hook
forecast/model.py	model_params, fit_models, predict_lines
forecast/postprocess.py	Sort, clip, closed days, dead_item_rule
forecast/experiments.py	keep_change (both-folds rule), log_experiment
forecast/review.py	compare_lines, worst_items, shuffle_importance, leak_check, group_check
forecast/tune.py · calibrate.py	≤ 6-setting grid on both folds · fit_shifts, cross_fold_check, apply_shifts
forecast/freeze.py · predict.py	Freeze + manifest + guards · predict(sales, origin) in contract columns
forecast/write_forecasts.py · report.py	56 origins → forecasts.parquet, forecasts_for · summary.md + charts

Useful commands (run from project/retailops in the Windows .venv)

```
python -m pytest tests -q # 163 tests
python -m forecast.evaluate # baseline and model scores on both folds
python -m forecast.freeze # retrain + freeze (laptop only)
python -m forecast.write_forecasts # write forecasts.parquet (laptop only)
python -m forecast.report # summary.md + charts
```

How well it works, and where it is weak

Validation scores (all four lines)

Pinball loss for each line (lower is better). The test window is not scored here (R10: only after the replay).

Fold	Method	MAE	q50	q80	q90	q95
Fold 1 festive	Naive	4.883	2.442	1.971	1.398	0.964
	Stub (4-week)	4.389	2.195	1.805	1.284	0.883
	Model	**3.931**	**1.966**	**1.729**	**1.225**	**0.800**
Fold 2 main	Naive	5.210	2.605	2.077	1.464	1.003
	Stub (4-week)	4.725	2.362	1.939	1.374	0.943
	Model	**4.149**	**2.075**	**1.723**	**1.201**	**0.778**

Coverage: does each line hold what it promises?

Fold	q50 (target 50%)	q80 (80%)	q90 (90%)	q95 (95%)
Fold 1 festive	54.8%	81.0%	89.9%	94.1%
Fold 2 main	52.8%	80.4%	89.9%	94.6%

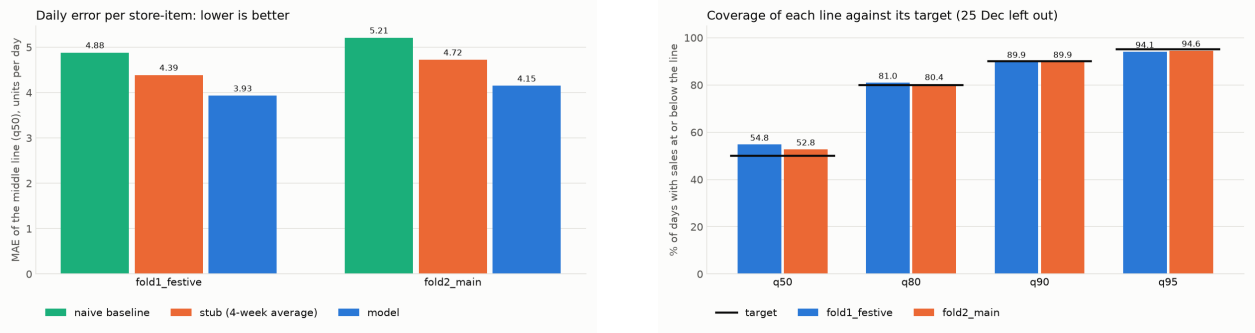


Figure 9. Error vs the baselines (left) and coverage of each line (right), from outputs/forecast/report.

Mean forecasts over the test file compared with the stub: q50 6.78 vs 7.92; q90 14.20 vs 14.08; q95 17.02 vs 15.84. Our spread is wider at the top, which is what the festive-day coverage needed.

Known limits (say these openly in the viva)

Limits and caveats

1. q50 covers 55% on fold 1, not 50%: the "0 = 0 ties". 8.3% of fold 1's rows are exact ties, mostly the dead-item rule forecasting 0 and 0 being sold; a tie counts as "covered". Counting ties as half gives 50.9% (fold 2: 50.0%). Gate D passed with this note.

2. Upper lines don't drop when an item suddenly stops selling. The dead-item rule zeroes only q50; q80–q95 stay high for weeks because the old spike stays in the 28-day spread. Example: CA_1 FOODS_3_090 sells 0 from 2 Dec 2015 while its q95 climbs from ~80 to ~180 a day. On validation, store-items with 15+ zero days were still stocked to about 2.0× (fold 1) / 1.1× (fold 2) their old average. A trade-off, not a bug: it lets a returning item get stock. Defences belong in the order model: a P2 days-of-supply cap on q50, the waste cost using q50, and a P3 flag for 14+ zero-sales days with stock on hand. Under the 7-day rule the forecaster cannot see a stop until 7+ days after it.

3. The example chart shows test-window sales. The report's example store-item chart (below) plots real sales from the test window. It was made for illustration after the forecasts were frozen and was not used to choose anything, but say so if asked.

4. Perishables in fold 2 got slightly worse after the stopped-selling fix (MAE 6.89 vs 6.77), while every line improved overall.

5. The baselines dip in early January because they still treat 25 Dec as a real near-zero day. No validation or test day is affected; they stay unchanged because they are the teammate's rule.

6. R2 costs accuracy for tomorrow (~12% for day 1, ~5% over the week) in exchange for one simple, provably fair model.

7. Laptop vs cloud differ in the third decimal (fold 1 q50 MAE 4.108 vs 4.106), which is why freezing is laptop only.

8. Perishable shelf lives (2–7 days) are an assumption: M5 items are anonymous.

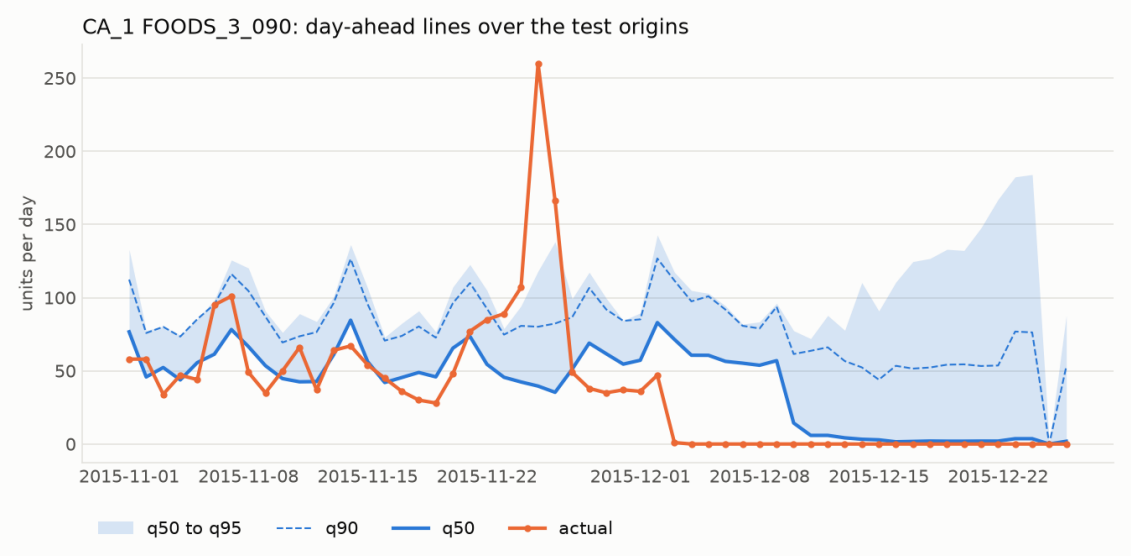


Figure 10. Example store-item over the test origins (limit 2 and disclosure 3): sales stop on 2 Dec, q50 falls to 0 a week later, but the q50–q95 band widens.

Likely viva questions, short answers

Q. How do you know the model doesn't cheat?

Every store input is at least 7 days old, checked by a registry and by tests that poison all future data with ×1000 values and confirm the inputs don't change. Windows are split by date, the holdout is sealed, statistics come from training rows only, and a shuffle test on the trained model found no leak (fold 2 MAE 4.15, well above the 3.6 alarm).

Q. Why not just forecast tomorrow?

Supplier lead time is 5 days, orders come in cases and trucks, perishables expire and festivals need build-up. We plan 7 days and act on 1.

Q. Why quantiles instead of one number?

Products are stocked to different safety levels (80/90/95%), so the order model needs those lines directly. A learned line beats mean + z × std because real sales are skewed and spiky.

Q. Why gradient-boosted trees?

They handle missing values and categories, train in minutes, need little tuning and are easy to explain. A neural network was considered and reserved.

Q. How do you know it's better?

It beats the teammate's 4-week rule by 10.4% and 12.2% in MAE on both folds, one festive, and every upper line beats both baselines in pinball loss.

Q. Why two validation folds?

Fold 2 has no festivals but the test window does. Fold 1 lets us check festive behaviour without touching the test window, and a change is kept only if it helps both.

Q. What did you try that didn't work?

Promo inputs, price vs department, a 52-week normal price, target-day price, recency weighting, more trees, log target, seed averaging, baseline blending. None gave a consistent gain on both folds.

Q. What would you do with more time?

A horizon-aware model (~5% better over the week), a 'days since last sale' input so upper lines fall for stopped items, and 5-day total forecasts for warehouse orders (layer C).

Glossary

Term	Meaning
Origin	The morning a forecast is made; covers origin ... origin+6
Quantile / line (q50...q95)	A level that real sales stay at or under a given share of the time
Pinball loss	The error measure that trains and scores a quantile line (unequal cost for short vs over)
MAE	Mean absolute error of q50, units per store-item-day
Coverage	Share of days with sales at or under a line; a good q90 covers ~90%
Fold	A validation period with its own training cut-off
Baseline / stub	Simple rules to beat: same day last week; teammate's 4-week weekday average
Service line	The line a product is stocked to: q95 perishables and top 30, q80 slow, q90 others
Dead-item rule	q50 = 0 when the 28-day average ending 7 days back is 0
Calibration (k)	Stretch of the gap above q50 if coverage is off by more than 3 points
Freeze / manifest	The saved final model and the record of exactly how it was made
Holdout	Sealed last ~5 months, run once after code freeze
Layer	An optional extra behind a config switch, default off